



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Патерни проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	3
Хід роботи	5
Діаграма класів патерну Composite	5
Код програми	5
Висновок	10
Питання до лабораторної роботи	10

Теоретичні відомості

Шаблон «Composite»

Шаблон використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю.

Простим прикладом може служити складання компонентів всередині звичайної форми. Форма може містити дочірні елементи (поля для введення тексту, цифр, написи, малюнки тощо); дочірні елементи можуть в свою чергу містити інші елементи. Наприклад, при виконанні операції розтягування форми необхідно, щоб вся ієрархія розтягнулася відповідним чином. В такому випадку форма розглядається як композитний об'єкт і операція розтягування застосовується до всіх дочірніх елементів рекурсивно.

Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів. Крім того, патерн «Composite» (Компонувальник) краще використовувати, коли ви представляєте структуру даних у вигляді дерева.

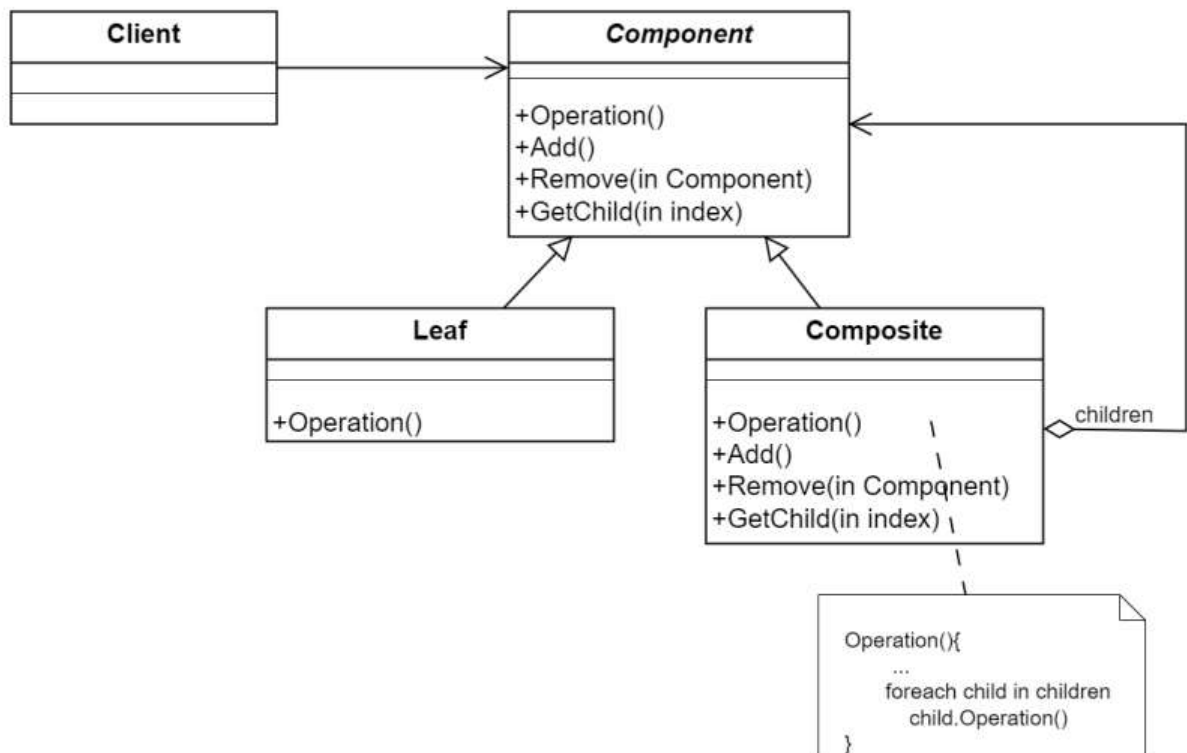


Рис. 1. Структура патерна «Компонувальник»

Проблема: Ви розробляєте систему керування проектами. Кожен проект складається із наборів функцій, кожна функція з userstory, а кожна userstory в свою чергу із задач по її реалізації. Ви реалізуєте функціонал відображення оціночної вартості робіт по кожній із функцій, а також відображення чи всі

userstory були оцінені. Це потрібно бізнес-аналітики, щоб розуміти, що всі userstory розробниками були розглянуті і оцінені, а також обговорити необхідність реалізації того чи іншого функціоналу на основі попередньої оцінки.

Рішення: Крашим підходом в даній ситуації буде використання патерну Компоновщик. Класи що представляють функції, userstory, задачі будуть наслідуватися від одного інтерфейсу ITask, функції (Feature) та Userstory будуть складними об'єктами і міститимуть колекції об'єктів ITask, а задачі (Task) будуть представляли кінцеві об'єкти без дочірніх елементів.

Для розрахунку оціночної вартості робіт, в ITask інтефейс додаємо метод GetEstimatedPoints(). В класах-компоновщиках методи GetEstimatedPoints() реалізовуємо як обхід всіх дочірніх елементів та сумування результатів відповідей GetEstimatedPoints().

Таким чином, візуальні форми будуть працювати з колекцією елементів ITask і їм не потрібно буде знати конкретні типи дочірніх класів з якими вони працюють. В результаті логіка візуальних форм виходить достатньо простою і вона не буде містити бізнес-логіки розрахунку загальної оцінки по проєктам, а просто викликає метод GetEstimatedPoints() не замислюючись містить цей об'єкт дочірні об'єкти чи ні.

Переваги та недоліки:

- + Спрощує представлення деревоподібної структури.
- + Додає гнучкості в роботі з складними об'єктами та рекурсивними операціями.
- + Дозволяє додавати та видаляти об'єкти в ієрархії без впливу на клієнтський код.
- Потрібні додаткові зусилля для початкового впровадження.
- Вимагає гарно спроектованого загального інтерфейсу.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів патерну Composite

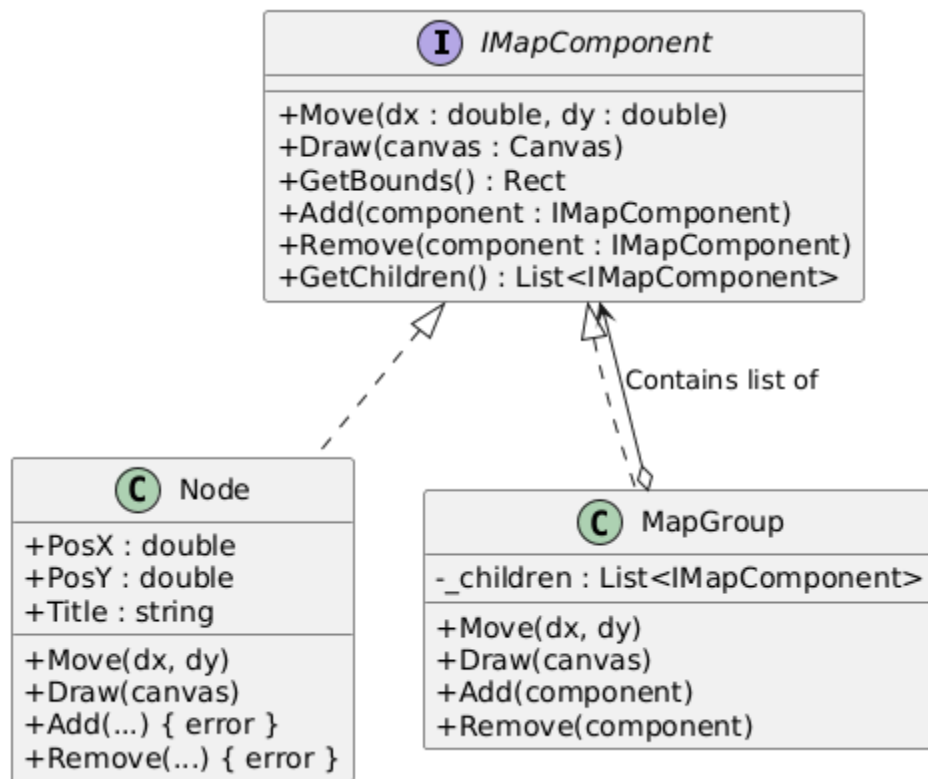


Рис. 2. Діаграма класів патерну Composite

Код програми

Region (оновлено)

```
using System.Collections.Generic;
using System.Windows;
using System.Linq;
```

```
namespace MindMapApp.Entities
{
    public class Region : IMapComponent
    {
        public int Id { get; set; }
        public string Title { get; set; } = string.Empty;
        public string BorderColor { get; set; } = "#FF0000";

        public int MindMapId { get; set; }
        public virtual MindMap MindMap { get; set; }

        public virtual ICollection<Node> Nodes { get; set; } = new List<Node>();
        public void Move(double dx, double dy)
        {
            if (Nodes == null) return;

            foreach (var node in Nodes)
            {
                node.Move(dx, dy);
            }
        }
        public Rect GetBounds()
        {
            if (Nodes == null || Nodes.Count == 0) return Rect.Empty;
            Rect totalBounds = Nodes.First().GetBounds();
            foreach (var node in Nodes.Skip(1))
            {
                totalBounds.Union(node.GetBounds());
            }
        }
    }
}
```

```

    }
    totalBounds.Inflate(10, 10);

    return totalBounds;
}
}
}

```

MainWindow (додано методи)

```

    private void DrawingCanvas_MouseDown(object sender,
    MouseButtonEventArgs e)
    {
        if (e.ChangedButton == MouseButton.Middle && e.ButtonState ==
    MouseButtonState.Pressed)
        {
            if (e.OriginalSource is FrameworkElement element &&
    element.DataContext is Node node)
            {
                _isDragging = true;
                _selectedNode = node;
                _lastMousePosition = e.GetPosition(DrawingCanvas);

                DrawingCanvas.CaptureMouse();
                e.Handled = true;
            }
        }
    }

    private void DrawingCanvas_MouseMove(object sender, MouseEventArgs e)
    {
        if (_isDragging && _selectedNode != null)
        {

```

```

        if (e.MiddleButton == MouseButtonState.Pressed)
        {
            Point currentPoint = e.GetPosition(DrawingCanvas);
            double dx = currentPoint.X - _lastMousePosition.X;
            double dy = currentPoint.Y - _lastMousePosition.Y;
            if (_selectedNode.Region != null)
            {
                _selectedNode.Region.Move(dx, dy);
            }
            else
            {
                _selectedNode.Move(dx, dy);
            }
            _lastMousePosition = currentPoint;
            DrawCurrentMap();
        }
    }

    private void DrawingCanvas_MouseUp(object sender,
    MouseButtonEventArgs e)
    {
        if (e.ChangedButton == MouseButton.Middle)
        {
            if (_isDragging)
            {
                _isDragging = false;
                _selectedNode = null;
                DrawingCanvas.ReleaseMouseCapture();
                Repository.Update(CurrentMap);
            }
        }
    }

```



```

        }
    }
}

private void BtnCreateRegion_Click(object sender, RoutedEventArgs e)
{
    if (CurrentMap == null) return;

    var freeNodes = CurrentMap.Nodes.Where(n => n.RegionId ==
null).ToList();

    if (freeNodes.Count < 2)
    {
        MessageBox.Show("Треба хоча б 2 вільних вузли для створення
групи!");
        return;
    }

    var newRegion = new MindMapApp.Entities.Region
    {
        Title = "Нова група",
        BorderColor = "#0000FF",
        MindMapId = CurrentMap.Id,
        Nodes = freeNodes
    };

    CurrentMap.Regions.Add(newRegion);
    Repository.Update(CurrentMap);
    CurrentMap = Repository.GetById(CurrentMap.Id);
    DrawCurrentMap();
}
}
}

```

IMapComponent (створено)

using System.Windows;

```
namespace MindMapApp.Entities
{
    public interface IMapComponent
    {
        void Move(double dx, double dy);

        Rect GetBounds();
    }
}
```

Висновок: Під час виконання роботи було імплементовано патерн Composite завдяки цьому було створити механізм групування вузлів, що дозволило взаємодіяти з групою так само як з окремим елементом.

Питання до лабораторної роботи

1. Яке призначення шаблону «Композит»?
Дозволити клієнтам трактувати окремі об'єкти та їх групи (деревоподібні структури) однаковим чином, використовуючи спільний інтерфейс як для простих, так і для складених елементів ієрархії.
2. Які класи входять в шаблон «Композит», та яка між ними взаємодія?
Абстрактний клас Компонент, прості елементи та складені елементи. Взаємодія полягає в тому, що Компонент оголошує інтерфейс, простий елемент виконує реальну роботу, а складний елемент делегує виконання запиту своїм дочірнім елементам, які можуть бути як листками, так і іншими контейнерами.
3. Яке призначення шаблону «Легковаговик»?
Ефективна підтримка великої кількості дрібних об'єктів шляхом розділення їхнього стану на внутрішній та зовнішній, що дозволяє економити оперативну пам'ять завдяки повторному використанню спільних даних.
4. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?
Інтерфейс Легковаговика, Конкретний легковаговик, Фабрика легковаговиків та Клієнт. Взаємодія відбувається через Фабрику, яка керує пулом об'єктів і повертає існуючий екземпляр або створює

новий, а Клієнт передає унікальний зовнішній стан у методи Легковаговика лише в момент виконання операції.

5. Яке призначення шаблону «Інтерпретатор»?

Визначення граматики простої мови та створення інтерпретатора, який використовує це представлення для розбору та виконання речень цієї мови, що часто використовується для обробки виразів або створення DSL.

6. Яке призначення шаблону «Відвідувач»?

Додавання нових операцій над об'єктами складної структури без зміни класів цих об'єктів, що дозволяє відокремити алгоритм від структури даних, над якою він працює.

7. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

Інтерфейс Відвідувача, Конкретні відвідувачі, інтерфейс Елемента (з методом Асепт) та Конкретні елементи; взаємодія базується на подвійній диспетчеризації: клієнт викликає метод Асепт на Елементі, передаючи йому Відвідувача, а Елемент викликає відповідний метод Visit у Відвідувача, передаючи посилання на себе.